

UNIT 5: DECIDABILITY AND COMPUTATIONAL COMPLEXITY

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Comprehensive Master Theory Guide	Course Code: PCC-303-ITT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 5.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Decidability & Language Classes	Church-Turing Thesis, Recursive Languages (Decidable), Recursively Enumerable Languages (Semi-decidable), Complement closure properties theorem.	High Priority (6-8 Marks Qs)
Decidable & Undecidable Problems	Decidable problems for Regular & Context-Free Languages (Emptiness, Finiteness, Membership), Undecidability definition, Halting Problem proof (H_{TM}).	High Priority (8-10 Marks Qs)
Reducibility & Post Correspondence	Mapping Reducibility ($L_1 \leq L_2$), Post Correspondence Problem (PCP) & Modified PCP (MPCP) statement & proof of undecidability.	High Priority (8-10 Marks Qs)
Computational Complexity Classes	Time Complexity $T(n)$, Class P (Polynomial Time), Class NP (Nondeterministic Polynomial Time), Polynomial-time reducibility ($\leq_p$).	High Priority (8-10 Marks Qs)
NP-Completeness & Classic Problems	Definition of NP-Hard and NP-Complete, Cook-Levin Theorem (Boolean Satisfiability - SAT), 3-SAT, Vertex Cover, Clique, Traveling Salesperson Problem.	High Priority (8-10 Marks Qs)

Church-Turing Thesis and Decidability Foundations

The Church-Turing Thesis

The Church-Turing Thesis posits that the intuitive, informal notion of an effective algorithm or computation is precisely captured by the mathematical model of a Turing Machine.

DEFINITION:

Church-Turing Thesis: Any algorithmic process that can be performed by a human or digital computer can be simulated by a Turing Machine.

Recursive vs. Recursively Enumerable Languages

DEFINITION:

1. **Recursive Language (Decidable Language):** A language $L \subseteq \Sigma^*$ for which there exists a Turing Machine M that **halts on all inputs** $w \in \Sigma^*$:
 - If $w \in L$, M halts in an accepting state.
 - If $w \notin L$, M halts in a rejecting state.
2. **Recursively Enumerable (RE) Language (Turing-Recognizable):** A language L for which there exists a Turing Machine M such that:
 - If $w \in L$, M halts in an accepting state.
 - If $w \notin L$, M either halts in a non-accepting state OR loops infinitely.

IMPORTANT:

Complement Closure Theorem: A language L is **Recursive** if and only if both L and its complement L^c are **Recursively Enumerable (RE)**.

Proof Outline: Run TMs M_1 (recognizing L) and M_2 (recognizing L^c) in parallel step-by-step. Since every string w belongs to either L or L^c , exactly one TM will halt and accept, guaranteeing a total decider algorithm.

Source: Hopcroft et al., Ch 9, pp. 375–385 | Source: Sipser, Ch 3, pp. 195–210

Decidable and Undecidable Problems

A decision problem is a question with a binary Yes/No answer over a set of inputs.

Decidability of Formal Language Properties

Decision Problem	Regular	Context-Free	Recursively Enumerable
Membership Problem ($w \in L$)	Decidable	Decidable (CYK)	Undecidable (Halting)
Emptiness Problem ($L = \emptyset$)	Decidable	Decidable	Undecidable
Finiteness Problem ($ L < \infty$)	Decidable	Decidable	Undecidable
Equivalence ($L_1 = L_2$)	Decidable	Undecidable	Undecidable
Ambiguity of Grammar	N/A	Undecidable	N/A

The Halting Problem of Turing Machines

DEFINITION:

Statement of Halting Problem (H_{TM}): Given a Turing Machine M and an input string w , determine whether M eventually halts when run on w .

$$H_{TM} = \{(M, w) : M \text{ is a TM that halts on input } w\}$$

NOTE:**Proof of Undecidability via Diagonalization:**

1. Assume by contradiction that there exists a decider TM $H(M, w)$ that outputs “YES” if M halts on w , and “NO” if M loops forever on w .
2. Construct a new machine $D(M)$ that takes a string representation of TM M :
 - D calls $H(M, M)$.
 - If H returns “YES” (halts), D enters an infinite loop.
 - If H returns “NO” (loops), D halts immediately.
3. Now run D on its own description: $D(D)$.
 - If $D(D)$ halts, $H(D, D)$ outputs “YES” $\implies D(D)$ loops infinitely (Contradiction).
 - If $D(D)$ loops, $H(D, D)$ outputs “NO” $\implies D(D)$ halts (Contradiction).
4. Hence, no such decider H can exist. The Halting Problem is **Undecidable**.

Source: Hopcroft et al., Ch 9, pp. 386–398 | Source: KLP Mishra, Ch 10, pp. 320–335

Reducibility and Post Correspondence Problem (PCP)

Reducibility is the primary technique for proving new problems undecidable by transforming known undecidable problems.

Mapping Reducibility ($L_1 \leq_m L_2$)

A language L_1 is mapping reducible to L_2 ($L_1 \leq_m L_2$) if there exists a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for all $w \in \Sigma^*$:

$$w \in L_1 \iff f(w) \in L_2$$

If L_1 is undecidable and $L_1 \leq_m L_2$, then L_2 is also **undecidable**.

Post Correspondence Problem (PCP)**DEFINITION:**

Post Correspondence Problem (PCP): Given two lists of strings of equal length over alphabet Σ :

$$A = (w_1, w_2, \dots, w_k) \quad \text{and} \quad B = (x_1, x_2, \dots, x_k)$$

Is there a sequence of indices $i_1, i_2, \dots, i_m$ ($1 \leq i_j \leq k, m \geq 1$) such that:

$$w_{\{i_1\}} w_{\{i_2\}} \dots w_{\{i_m\}} = x_{\{i_1\}} x_{\{i_2\}} \dots x_{\{i_m\}}$$

Theorem: PCP is **undecidable** in the general case (proved by reducing TM computation histories to PCP instances).

Source: Hopcroft et al., Ch 9, pp. 401–412 | Source: KLP Mishra, Ch 10, pp. 336–345

Computational Complexity Classes (P, NP, NP-Complete)

Complexity theory measures the time and space resources required by algorithms to solve decidable problems.

Time Complexity and Class P

DEFINITION:

1. **Time Complexity** $T(n)$: The maximum number of transition steps taken by a deterministic TM M on any input of length n .
2. **Class P**: The class of all decision problems solvable by a Deterministic Turing Machine in **polynomial time** $O(n^k)$ for some constant k .
 - Examples: Shortest path (Dijkstra), Minimum Spanning Tree (Kruskal), DFA State Minimization, Parsing CNF (CYK).

Class NP and Polynomial-Time Reducibility

DEFINITION:

1. **Class NP (Nondeterministic Polynomial Time)**: The class of decision problems solvable by a Nondeterministic Turing Machine in polynomial time, OR equivalently, problems whose candidate solutions can be **verified** in polynomial time by a Deterministic TM.
2. **Polynomial-Time Reduction** ($L_1 \leq_p L_2$): A language L_1 is polynomial-time reducible to L_2 if there exists a polynomial-time computable function f such that $w \in L_1 \iff f(w) \in L_2$.

NP-Hardness and NP-Completeness

DEFINITION:

1. **NP-Hard**: A language L is NP-Hard if for **every** language $L' \in \text{NP}$, $L' \leq_p L$.
2. **NP-Complete**: A language L is NP-Complete if:
 - $L \in \text{NP}$, AND
 - L is NP-Hard ($L' \leq_p L$ for all $L' \in \text{NP}$).

IMPORTANT:

Cook-Levin Theorem: The Boolean Satisfiability Problem (**SAT**) is **NP-Complete**. It was the first problem proven to be NP-complete by simulating Nondeterministic Polynomial-Time TMs using logical Boolean formulas.

Classic NP-Complete Problems

1. **SAT & 3-SAT**: Boolean formula satisfiability.
2. **Vertex Cover Problem**: Finding a set of k vertices touching all graph edges.
3. **Clique Problem**: Finding a complete subgraph of size k .
4. **Hamiltonian Cycle & Traveling Salesperson Problem (TSP)**.

Source: Hopcroft et al., Ch 10, pp. 425–455 | Source: Sipser, Ch 7, pp. 275–310